



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

{ FACHBEREICH
INFORMATIK }

Programmierung 1

Aufbau von Programmiersprachen

WS 2026/27

Bachelor Cyber Security & Privacy (B.Sc.)

André Kless, M.Sc.

Rückblick: Algorithmen selbst formulieren

Sie können kleine algorithmische Probleme lösen –

unabhängig von einer konkreten Programmiersprache,
mit den Grundbausteinen und in einer klaren Darstellung.

Grundbausteine

Einzelanweisung, Sequenz, Selektion,
Mehrfachselektion und Iteration

Darstellung

Pseudocode
oder Struktogramm

Unser Pseudocode ist bereits eine kleine Sprache für Algorithmen.

Unser Ziel: die eigene Sprache „Algo“

Der bekannte ggT-Algorithmus als Ausgangspunkt

```
DEKLARIERE a, b
LIES a EIN
LIES b EIN
SOLANGE a ≠ b:
    WENN a > b:
        SETZE a AUF a - b
    SONST:
        SETZE b AUF b - a
    ENDE WENN
ENDE SOLANGE
GIB a AUS
```

Algo

Unsere eigene Sprache
für Algorithmen.

Als Werte erlauben wir
nur Ganzzahlen.
So bleibt Algo einfach.

$a, b > 0$
18 und 12: Ausgabe 6

Am Ende können Sie eine Grammatik für einfache Algorithmen entwerfen.

Definition: Programmiersprache

“artificial language for expressing programs”

Quelle: ISO/IEC/IEEE 24765:2017 (Systems and software engineering — Vocabulary),
Eintrag 3.3147, Bedeutung 2

Eigene Übersetzung

Eine künstliche Sprache zum Ausdrücken von Programmen.

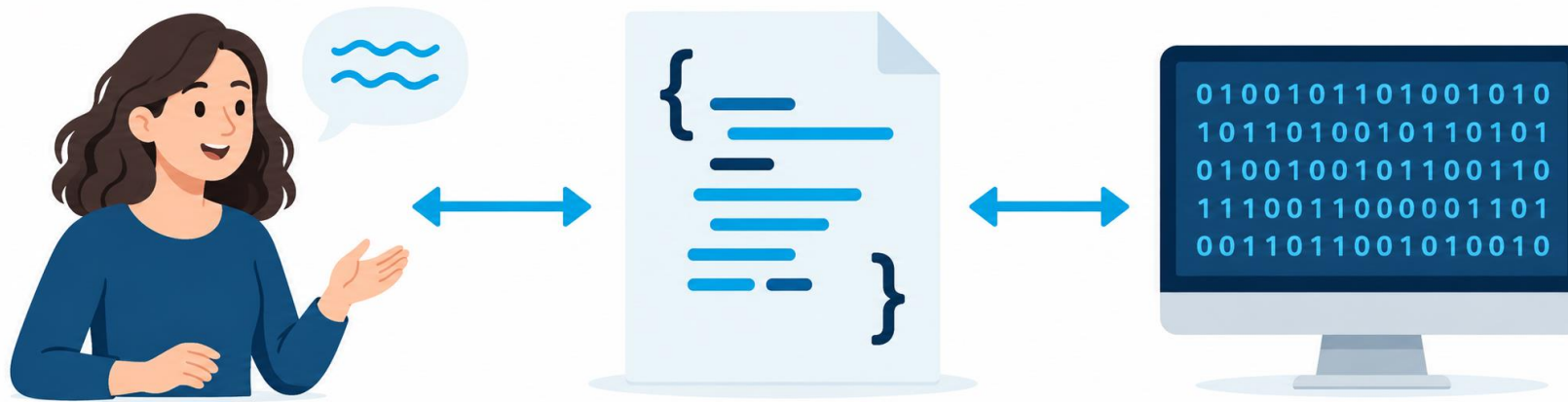
Ihre Regeln werden vor der Verwendung ausdrücklich festgelegt.

Wozu wir Programmiersprachen brauchen

Mensch

Programmiersprachen

Computer



Natürliche Sprache:
flexibel, oft mehrdeutig.

Kompromiss: gut lesbar
und eindeutig verarbeitbar.

Maschinenbefehle:
präzise, schwer zu lesen.

Unser Pseudocode ist gut lesbar. Seine eindeutigen Regeln ermöglichen auch die Verarbeitung durch ein Programm.

Algo-Text auf Schreibregeln prüfen

Der ggT-Algorithmus

```
DEKLARIERE a, b
LIES a EIN
LIES b EIN
SOLANGE a ≠ b:
    WENN a > b:
        SETZE a AUF a - b
    SONST:
        SETZE b AUF b - a
    ENDE WENN
ENDE SOLANGE
GIB a AUS
```

Ein Wert ist hier eine ganze Zahl oder ein Bezeichner.

Die Schreibregeln legen fest, welche Formen von Anweisungen zulässig sind.

Was die Schreibregeln verlangen

- 1 Nach **DEKLARIERE** folgt ein Bezeichner. Beliebig viele weitere sind erlaubt, jeweils mit Komma und Leerzeichen.
- 2 Nach **LIES**: Bezeichner und **EIN**. Nach **GIB**: ein Wert und **AUS**.
- 3 Nach **SETZE**: Bezeichner und **AUF**. Dann ein Wert oder zwei mit Operator.
- 4 Nach **SOLANGE** / **WENN**: Vergleich und **:**. Passende Endzeilen, ggf. **SONST** **:**.
- 5 Pro Block zwei Leerzeichen mehr. Bezeichner: nur Buchstaben von **a** bis **z**.

Vom Algo-Text zu ausführbaren Schritten

Der ggT-Algorithmus

```
DEKLARIERE a, b
LIES a EIN
LIES b EIN
SOLANGE a ≠ b:
    WENN a > b:
        SETZE a AUF a - b
    SONST:
        SETZE b AUF b - a
    ENDE WENN
ENDE SOLANGE
GIB a AUS
```

Voraussetzung: **a** und **b** sind positive ganze Zahlen.

Die Sprachregeln legen fest, welche Wirkung jede Anweisung hat.

Bedeutung bei der Ausführung

- 1 Zwei Speicherplätze für ganze Zahlen unter den Namen **a** und **b** bereitstellen.
- 2 Zwei ganze Zahlen einlesen und in **a** beziehungsweise **b** speichern.
- 3 **a ≠ b** prüfen. Bei Gleichheit folgt direkt die Ausgabe.
- 4 Bei **a > b**: **a - b** in **a** speichern. Sonst **b - a** in **b**. Dann erneut prüfen.
- 5 Den gemeinsamen Wert **a** ausgeben.

Definitionen: Syntax und Semantik

Syntax: die zulässige Form

Die Syntax legt fest, welche Folgen von Symbolen in einer Sprache zulässig sind.

Semantik: die festgelegte Bedeutung

Die Semantik legt fest, was syntaktisch gültige Ausdrücke und Anweisungen bedeuten und welche Wirkung sie haben.

SETZE anzahl AUF anzahl + 1

Wert lesen, 1 addieren,
Ergebnis speichern.

Jede Einzelanweisung steht in einer eigenen Zeile. Einrückung gehört zur Syntax.

Syntax und Semantik am Beispiel

Ziel: `anzahl` von 4 auf 5 erhöhen. Die Variable ist bereits deklariert.

Algo-Anweisung	Syntax	Semantik
<code>SETZE anzahl AUF anzahl + 1</code>	korrekt	korrekt fürs Ziel (Ergebnis: 5)
<code>SETZE anzahl AUF anzahl - 1</code>	korrekt	inkorrekt fürs Ziel (Ergebnis: 3)
<code>SETZE anzahl AUF anzahl +</code>	inkorrekt (Wert fehlt)	nicht definiert (Syntaxfehler)
<code>SETZE anzahl <u>anzahl</u> + 1</code>	inkorrekt (AUF fehlt)	nicht definiert (Syntaxfehler)

„Semantisch korrekt“ bezieht sich hier auf das Ziel der Aufgabe.
Bei falscher Syntax ist die Programmbedeutung nicht festgelegt.

Definition: Alphabet

Ein Alphabet ist eine endliche, nichtleere Menge von Symbolen.

$\Sigma = \{a_1, \dots, a_n\}$ mit $n \geq 1$ Σ heißt Sigma.

Binärzahlen $\Sigma_{\text{bin}} = \{0, 1\}$

Ganze Zahlen $\Sigma_{\text{dez}} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, -\}$

Deutsch (Auszug) $\Sigma_{\text{de}} = \{a, \dots, z, A, \dots, Z, \ddot{a}, \ddot{o}, \ddot{u}, \beta, \text{‡}, ., !, ?, \dots\}$

Algo
(vollständig) $\Sigma_{\text{algo}} = \{a, \dots, z, A, B, D, E, F, G, I, K, L, N, O, R, S, T, U, W, Z, 0, \dots, 9, +, -, *, /, <, =, >, \neq, \text{Komma}, :, \text{‡}, \text{↵}\}$

Komma bezeichnet das Trennzeichen in DEKLARIERE a, b.

‡ steht für ein Leerzeichen, ↵ für einen Zeilenumbruch.

Großbuchstaben: nur die für unsere Schlüsselwörter nötigen Zeichen.

DEKLARIERE, LIES, EIN, GIB, AUS, SETZE, AUF, SOLANGE, WENN, SONST, ENDE

Definition: Wort

Ein Wort w ist eine endliche Folge von Symbolen aus einem Alphabet Σ .

Der Fachbegriff „Wort“ meint eine ganze Zeichenfolge.
Sie kann auch Leerzeichen und Zeilenumbrüche enthalten.

Binäralphabet 101 oder 001

Ganzzahlalphabet -42 oder --

Deutsch "Ich lerne."

Ein ganzer Satz ist hier ein Wort.

Algo DEKLARIERE zahl
 LIES zahl EIN
 GIB zahl AUS

Ein vollständiges Programm ist hier ein Wort, einschließlich aller Leerzeichen und Zeilenumbrüche.

**Anführungszeichen grenzen den dargestellten Text ab und gehören nicht zum Wort.
Ob ein Wort zu einer bestimmten Sprache gehört, ist eine weitere Frage.**

Konkatenation und das leere Wort

Konkatenation: zwei Wörter aneinanderhängen

$u = ab, \quad v = ba \quad uv = abba$

Das leere Wort ε enthält kein Symbol.

Verknüpfung	Neutrales Element	Es verändert nichts
Addition	0	$x + 0 = 0 + x = x$
Multiplikation	1	$x \cdot 1 = 1 \cdot x = x$
Konkatenation	ε	$u\varepsilon = \varepsilon u = u$

**Beispiel: $ab\varepsilon = \varepsilon ab = ab$. Das leere Wort fügt kein Symbol hinzu.
Ein Leerzeichen ist ein Symbol und deshalb kein leeres Wort.**

Definition: alle bildbaren Wörter

Σ^* ist die Menge aller endlichen Wörter über Σ , induktiv definiert:

1. $\varepsilon \in \Sigma^*$ Das leere Wort gehört dazu.
2. $a \in \Sigma \Rightarrow a \in \Sigma^*$ Jedes Symbol ist auch ein Wort.
3. $x, y \in \Sigma^* \Rightarrow xy \in \Sigma^*$ Wörter lassen sich aneinanderhängen.

Σ^* enthält genau die Wörter, die durch endlich viele Anwendungen dieser Regeln entstehen.

Binäralphabet $\Sigma_{\text{bin}}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$

Ganzzahlalphabet $\Sigma_{\text{dez}}^* = \{\varepsilon, 0, 1, -, 00, --, -42, \dots\}$

Deutsch $\Sigma_{\text{de}}^* = \{\varepsilon, "a", "aa", "??", "Ich lerne.", \dots\}$

Algo $\Sigma_{\text{algo}}^* = \{\varepsilon, "a", ":::", "GIB", "GIB1AUS↵", \dots\}$

Die Anführungszeichen grenzen den Text ab. Sie gehören nicht zum dargestellten Wort.

Definition: Länge eines Wortes

Die Länge $|w|$ ist die Anzahl der Symbole in einem Wort w .

Für das leere Wort gilt $|\epsilon| = 0$.

Anführungszeichen grenzen den Text ab und zählen nicht mit.
Leerzeichen und Zeilenumbrüche zählen als Symbole.

Beispiel	Wort w	Länge als Formel
Binäralphabet	101	$ 101 = 3$
Ganzzahlalphabet	-42	$ -42 = 3$
Deutsch	"Ich lerne."	$ "Ich lerne." = 10$
Algo	"GIB ₁ AUS"	$ "GIB1AUS" = 10$

Definition: formale Sprache

Eine formale Sprache L ist eine Menge von Wörtern über einem Alphabet Σ .

$$L \subseteq \Sigma^*$$

L ist eine Teilmenge aller bildbaren Wörter.

Binärzahlen

$$L_{\text{bin}} = \{0, 1, 10, 11, 100, 101, 110, \dots\} \subseteq \Sigma_{\text{bin}}^*$$

Zahlen im Zweiersystem: nur 0 und 1, ohne führende Nullen.

Ganze Zahlen

$$L_{\text{dez}} = \{0, 1, -1, 2, -2, 3, -3, \dots\} \subseteq \Sigma_{\text{dez}}^*$$

Wie aus der Schule: Dezimalziffern, ggf. Minus, ohne führende Nullen.

Deutsch

$$L_{\text{de}} = \{\text{"Ich lerne."}, \text{"Du lernst."}, \dots\} \subseteq \Sigma_{\text{de}}^*$$

Algo

$$L_{\text{algo}} = \{\text{"GIB0AUS$"}, \text{"GIB$1AUS"}, \dots\} \subseteq \Sigma_{\text{algo}}^*$$

Die einzelne 0 ist erlaubt. Führende Nullen ändern den Wert nicht, sind hier aber ausgeschlossen.

Von Aufzählungen zu genauen Regeln

$$L_{\text{bin}} = \{0, 1, 10, 11, 100, 101, \dots\}$$

Bisher haben wir die Fortsetzung nur mit „...“ angedeutet.

Das Muster ist intuitiv verständlich, aber noch nicht exakt festgelegt.

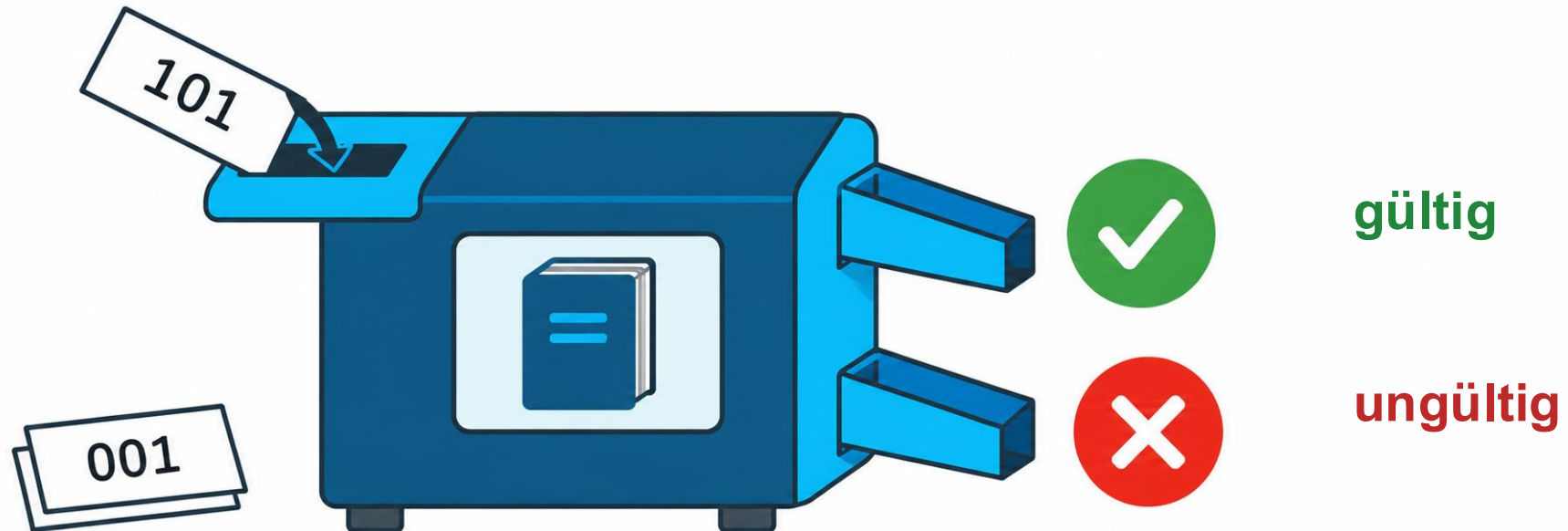
Wie legen wir genau fest, welche Wörter zur Sprache gehören und welche nicht?

Mit einer Grammatik.

Grammatik und Syntaxprüfung

Ein Syntaxprüfer prüft anhand der Grammatik,
ob eine eingegebene Zeichenfolge zur Sprache gehört.

Eingabe



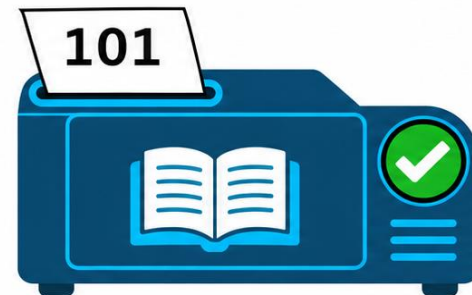
Beispiel: Binärzahlen ohne führende Nullen. 0 ist ebenfalls erlaubt.

Sprachprüfung an konkreten Wörtern

Binärzahlen

$101 \in L_{\text{bin}}$

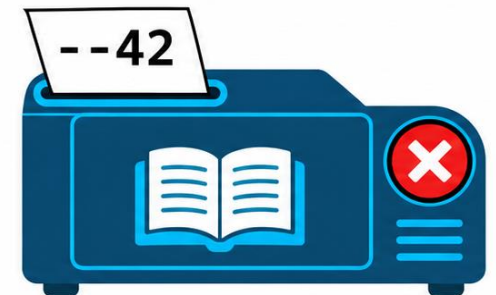
Beginnt mit 1.
Nur die Ziffern 0 und 1.



Ganze Zahlen

$--42 \notin L_{\text{dez}}$

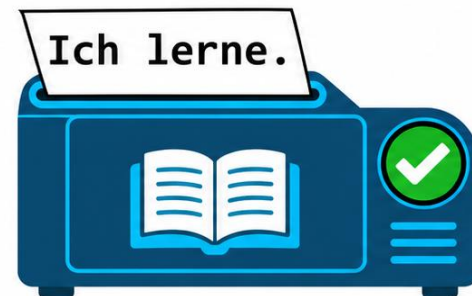
Zwei Minuszeichen
sind nicht erlaubt.



Deutsch

$\text{"Ich lerne."} \in L_{\text{de}}$

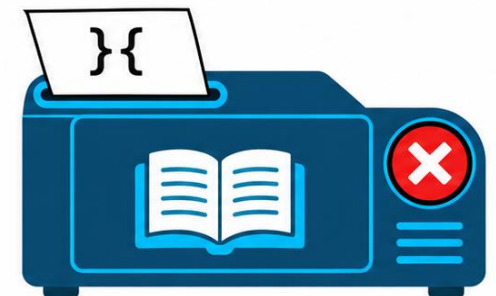
Ein syntaktisch
gültiger deutscher Satz.



Algo

$\text{"}\{\}\text{"} \notin L_{\text{algo}}$

Die Zeichen { und }
gehören nicht zum Algo-
Alphabet.



Definition: Chomsky-Grammatik

$$G = (\Sigma, N, P, S)$$

$$\Sigma \cap N = \emptyset \quad \text{und} \quad \Sigma \neq \emptyset$$

Bestandteil	Bedeutung
Σ	Endliche Menge der Terminalsymbole
N	Endliche Menge der Nichtterminale (selbst gewählte Hilfssymbole)
P	Endliche Menge der Produktionen (Ersetzungsregeln)
$S \in N$	Startsymbol (ein ausgewähltes Nichtterminal)

**Vom Startsymbol aus wenden wir Ersetzungsregeln an.
So erzeugen wir die Wörter der Sprache.**

**Zur Sprache gehören genau die erzeugbaren Wörter,
die nur noch aus Terminalsymbolen bestehen.**

Beispiel: eine Grammatik für Binärzahlen

Binärdarstellungen ohne führende Nullen. Die Zahl 0 ist erlaubt.

Grammatik

$$G = (\Sigma_{\text{bin}}, N, P, S)$$

$$\Sigma_{\text{bin}} = \{0, 1\}$$

$$N = \{X, R\}$$

$$S = X$$

$$P = \{ \begin{array}{l} X \rightarrow 0, \quad X \rightarrow 1R, \\ R \rightarrow 0R, \quad R \rightarrow 1R, \quad R \rightarrow \varepsilon \end{array} \}$$

$$L(G) = \{0, 1, 10, 11, 100, 101, 110, 111, \dots\}$$

Genau die aus X ableitbaren Wörter, die nur Terminale enthalten, gehören zur Sprache $L(G)$.

Konkrete Ableitungen

$$X \vdash 0$$

$$X \vdash 1R \vdash 10R \vdash 10$$

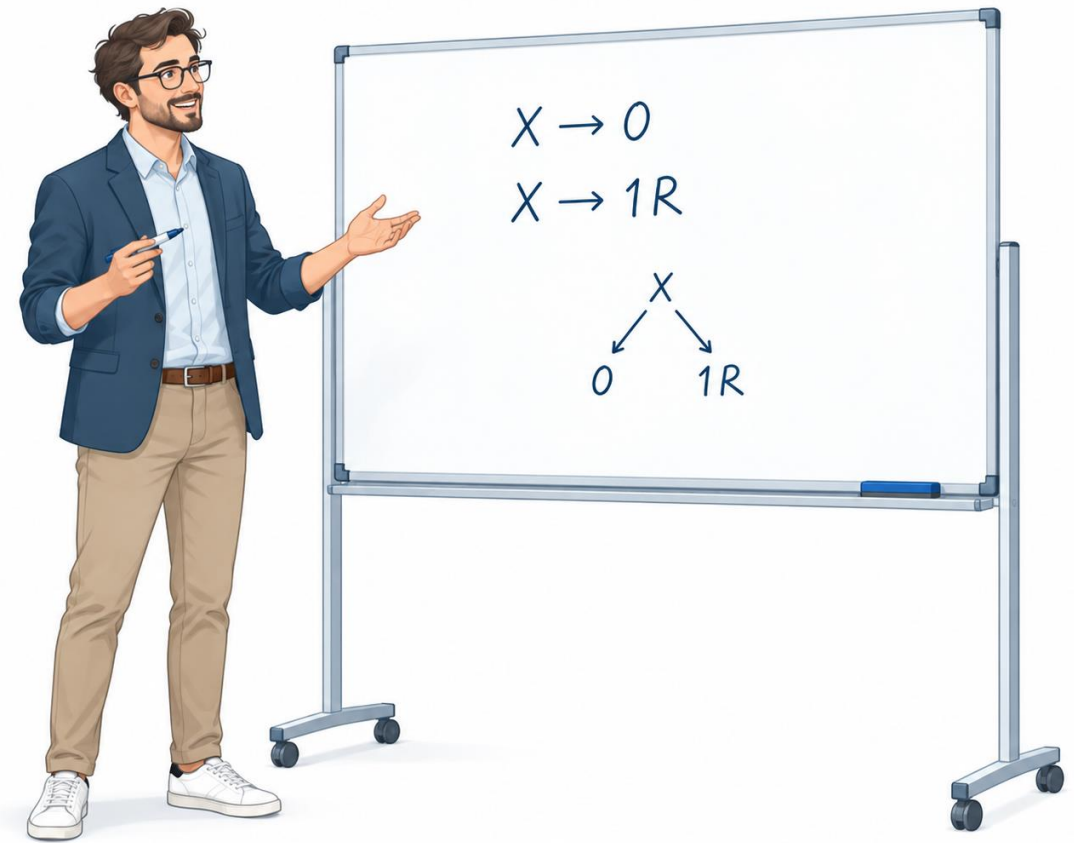
$$X \vdash 1R \vdash 10R \vdash 101R \vdash 101$$

$R \rightarrow \varepsilon$ entfernt das letzte Hilfssymbol.
Übrig bleiben nur 0 und 1.

Gemeinsames Beispiel am Whiteboard

Eine Grammatik entwickeln und Wörter ableiten

Die Regeln und Ableitungen entstehen gemeinsam am Whiteboard.



Definition: Ableitung

$\exists$: es existiert $\Leftrightarrow$: genau dann, wenn $:=$: ist definiert als

Sei $G = (\Sigma, N, P, S)$ eine Grammatik und $u, v \in (N \cup \Sigma)^*$.

- Ein Schritt** $u \vdash v \Leftrightarrow \exists x, y, y', z \in (N \cup \Sigma)^*:$
 $u = xyz, v = xy'z \text{ und } (y \rightarrow y') \in P$
Wir ersetzen y durch y' . Die Umgebung x und z bleibt erhalten.
- Genau n Schritte** $u \vdash^n v \Leftrightarrow \exists u_0, \dots, u_n \in (N \cup \Sigma)^* : u = u_0 \vdash \dots \vdash u_n = v$
Die Folge heißt Ableitung der Länge n . Dabei ist $n = 0, 1, 2, \dots$
- Beliebig viele** $u \vdash^* v \Leftrightarrow \exists n \in \mathbb{N}_0 : u \vdash^n v$
Null oder mehr Schritte sind erlaubt.
- Vom Startsymbol** $u \text{ ist in } G \text{ ableitbar} \Leftrightarrow S \vdash^* u$

$$L(G) := \{w \in \Sigma^* \mid S \vdash^* w\}$$

Alle vom Startsymbol ableitbaren Wörter, die nur Terminalsymbole enthalten.

Definition: Produktionsregeln

$$P \subseteq ((N \cup \Sigma^*) \setminus \Sigma^*) \times (N \cup \Sigma)^*$$

$$(N \cup \Sigma)^*$$

Alle endlichen Folgen aus Terminalen und Nichtterminalen.
Auch das leere Wort ε gehört dazu.

$$(N \cup \Sigma)^* \setminus \Sigma^*$$

Wir entfernen alle rein terminalen Wörter, auch ε .
Links bleibt mindestens ein Nichtterminal erforderlich.

$\times$

Das kartesische Produkt bildet Paare aus linker und rechter Seite.
Rechts ist jede Folge aus $N \cup \Sigma$ erlaubt, auch ε .

$(\alpha, \beta) \in P$ schreiben wir auch als $\alpha \rightarrow \beta$

$$P \subseteq \dots$$

P wählt eine endliche Menge solcher Paare als Regeln aus.

Chomsky-Hierarchie: Formen der Regeln

Von Typ 0 zu Typ 3: strengere Regeln, geringere Ausdrucksstärke, im Allgemeinen einfachere Syntaxprüfung.

Typ 0 $P \subseteq ((N \cup \Sigma)^* \setminus \Sigma^*) \times (N \cup \Sigma)^*$
unbeschränkt

Typ 1 Typ 0 und $\forall (\alpha, \beta) \in P: |\alpha| \leq |\beta|$
kontextsensitiv

Typ 2 $P \subseteq N \times (N \cup \Sigma)^*$
kontextfrei

Typ 3 $P \subseteq N \times (N\Sigma \cup \Sigma \cup \{\epsilon\})$ oder $P \subseteq N \times (\Sigma N \cup \Sigma \cup \{\epsilon\})$
regulär (linkslinear) (rechtslinear)

ϵ -Regeln sind ein Sonderfall des nichtverkürzenden Kriteriums.
Typ 1 hat eine Startsymbol-Ausnahme. Typ 2 und 3 erlauben $A \rightarrow \epsilon$.

Produktionsregeln nach Typ prüfen

Für alle Beispiele: $N = \{S, A, B\}$ und $\Sigma = \{a, b\}$.

Typ / Form	Zulässig für diese Form	Nicht zulässig für diese Form
Typ 0	$AB \rightarrow A$ links mindestens ein Nichtterminal	$ab \rightarrow A$ links kein Nichtterminal
Typ 1	$AB \rightarrow aB$ Länge bleibt gleich: $2 \leq 2$	$AB \rightarrow A$ Verkürzung: $2 > 1$
Typ 2 Unser Schwerpunkt	$A \rightarrow aAb$ links genau ein Nichtterminal	$AB \rightarrow ab$ links zwei Nichtterminale
Typ 3 linkslinear	$A \rightarrow Ba$ Nichtterminal steht links	$A \rightarrow aB$ rechtslinear statt linkslinear
Typ 3 rechtslinear	$A \rightarrow aB$ Nichtterminal steht rechts	$A \rightarrow Ba$ linkslinear statt rechtslinear

Alle Regeln prüfen: höchstmöglicher Typ und bei Typ 3 die gemeinsame Richtung.

Backus-Naur-Form: BNF

Dieselben Produktionsregeln, eine andere Schreibweise.

<code>:=</code>	Ersetzungszeichen anstelle von $\rightarrow$
<code><Name></code>	Spitze Klammern markieren ein Nichtterminal mit sprechendem Namen. Auch ein längerer Name bezeichnet ein einziges Hilfssymbol.
<code> </code>	ODER: Regeln mit gleicher linker Seite zusammenfassen.

$R \rightarrow 0$

$R \rightarrow 1$

`<Ziffer> := 0 | 1`

0 und 1 sind Terminalzeichen. `<Ziffer>` ist ein Nichtterminal.

Binärzahlen: Produktionsregeln und BNF

Bisher: Typ-2-Produktionen

$X \rightarrow 0$

$X \rightarrow 1R$

$R \rightarrow 0R$

$R \rightarrow 1R$

$R \rightarrow \varepsilon$

Startsymbol: X

Dieselben Regeln in BNF

$\langle \text{Binaerzahl} \rangle := 0 \mid 1 \langle \text{Rest} \rangle$

$\langle \text{Rest} \rangle := 0 \langle \text{Rest} \rangle \mid 1 \langle \text{Rest} \rangle \mid \varepsilon$

Startsymbol: $\langle \text{Binaerzahl} \rangle$

Gültig: 0, 1, 10, 101. Ungültig: 00, 01, 102. Die Sprache bleibt gleich.

Ganze Zahlen: Produktionsregeln und BNF

Typ-2-Produktionen

$X \rightarrow 0$ $X \rightarrow P$

$X \rightarrow -P$

$P \rightarrow ER$

$R \rightarrow ZR$ $R \rightarrow \varepsilon$

$Z \rightarrow 0$ $Z \rightarrow E$

$E \rightarrow 1\dots 9$

1...9 fasst hier neun Regeln zusammen.

Dieselben Regeln in BNF

$\langle \text{Ganzzahl} \rangle := 0 \mid \langle \text{Positiv} \rangle \mid - \langle \text{Positiv} \rangle$

$\langle \text{Positiv} \rangle := \langle \text{Nichtnullziffer} \rangle \langle \text{Rest} \rangle$

$\langle \text{Rest} \rangle := \langle \text{Ziffer} \rangle \langle \text{Rest} \rangle \mid \varepsilon$

$\langle \text{Ziffer} \rangle := 0 \mid \langle \text{Nichtnullziffer} \rangle$

$\langle \text{Nichtnullziffer} \rangle := 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Startsymbol: X bzw. $\langle \text{Ganzzahl} \rangle$. Gültig sind z. B. 0, 7, 105 und -42.

EBNF: Erweiterte Backus-Naur-Form

Alles aus BNF gilt weiterhin. Diese Klammern kommen hinzu:

Schreibweise	Bedeutung
[A]	A ist optional: null- oder einmal
{ A }	A wiederholen: null- oder mehrmals
(A B) C	Gruppieren: A oder B wählen, anschließend C

A, B und C stehen hier für Teile einer rechten Regelseite.
Die Klammern gehören zur Notation, nicht zum erzeugten Wort.

EBNF schreibt Regeln kürzer und beschreibt dieselben Sprachen wie BNF.

Binärzahlen: BNF und EBNF

BNF

`<Binaerzahl> := 0 | 1 <Rest>`

`<Rest> := 0 <Rest> | 1 <Rest> |  $\epsilon$`

Der Rückgriff auf `<Rest>`
beschreibt die Wiederholung.

EBNF

`<Binaerzahl> := 0 | 1 { 0 | 1 }`

`{ 0 | 1 }` ersetzt die Rest-Regel.
Null Wiederholungen sind erlaubt.

Startsymbol: `<Binaerzahl>`. Beide Beschreibungen erzeugen dieselben Wörter.

Ganze Zahlen: BNF und EBNF

BNF: fünf Hilfssymbole

`<Ganzzahl> := 0 | <Positiv> | -<Positiv>`

`<Positiv> := <Nichtnullziffer> <Rest>`

`<Rest> := <Ziffer> <Rest> | ε`

`<Ziffer> := 0 | <Nichtnullziffer>`

`<Nichtnullziffer> := 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

EBNF: drei Regeln

`<Ganzzahl> := 0 | [ - ]`

`<Nichtnullziffer> { <Ziffer> }`

`<Ziffer> := 0 | <Nichtnullziffer>`

`<Nichtnullziffer> := 1 | ... | 9`

1 | ... | 9 kürzt hier die neun Alternativen ab. Die Punkte gehören nicht zum Wort.

Syntaxdiagramme: die gesamte Notation

Ein gerichteter Weg erzeugt ein Wort. Jedes Nichtterminal hat ein Teildiagramm.

Terminal



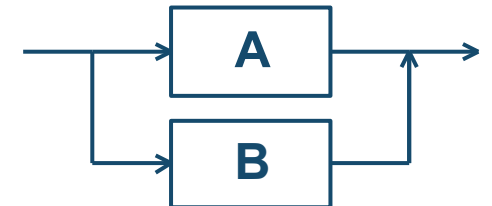
Nichtterminal



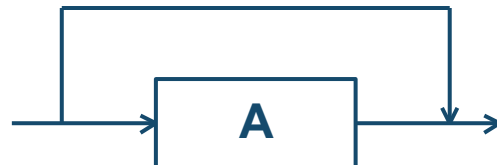
Folge: A B



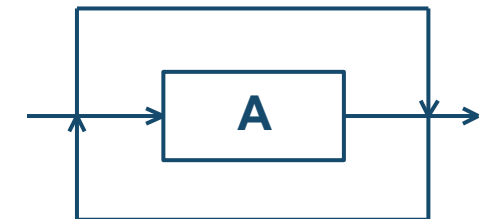
Alternative: A | B



Optional: [A]



Wiederholung: { A }



Runde Klammern gruppieren Wege. Die Pfeile bestimmen die Leserichtung.

Algo: <Anweisung> als Startsymbol

<Anweisung>



Verbindliche Schreibweise

`_` = ein Leerzeichen

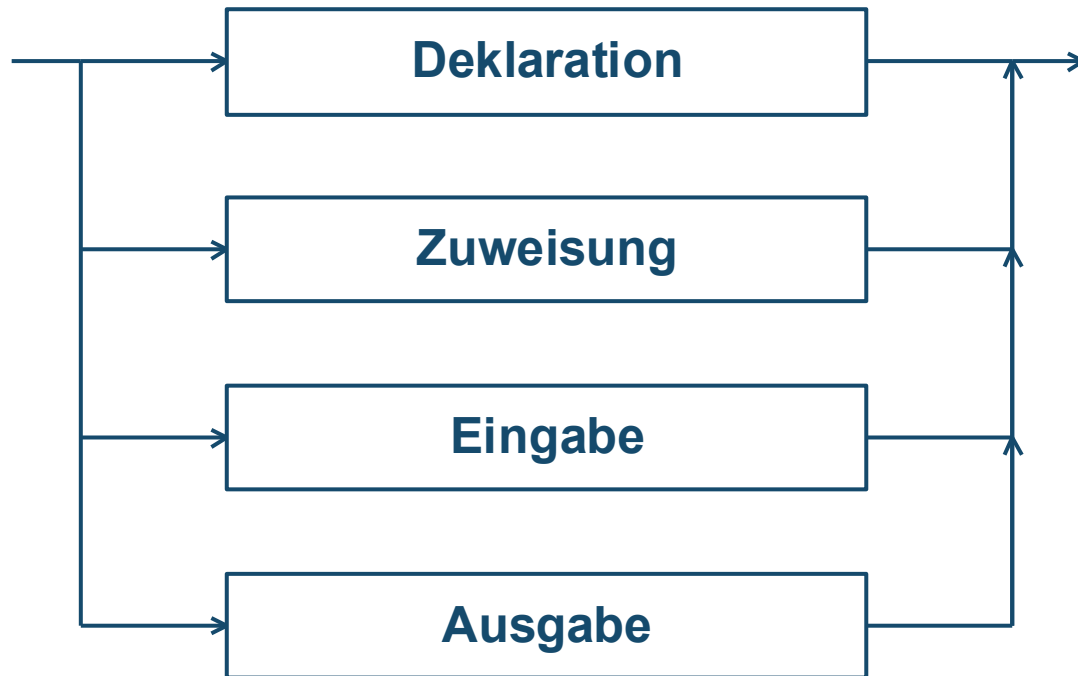
`↵` = ein Zeilenumbruch

Ein Programm beginnt ohne Einrückung.
Jeder Rumpf steht zwei Leerzeichen tiefer.
Das gilt für jede Zeile des Rumpfs.

**Die Einrückung ist eine zusätzliche
Formatregel zu den Diagrammen.**

Ein Programm ist eine Anweisung. Auch eine ganze Sequenz ist eine Anweisung.

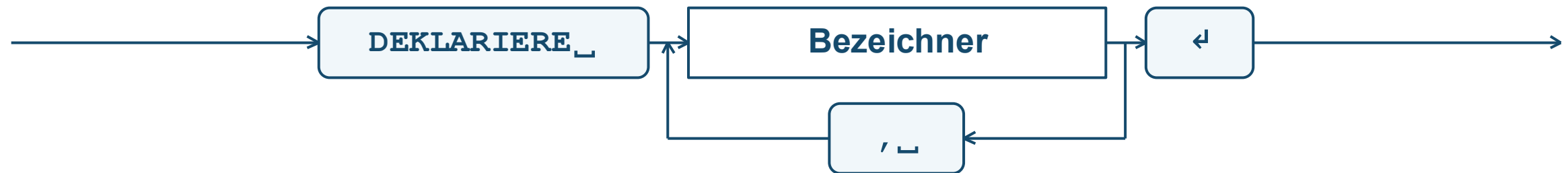
Algo: <Einzelanweisung>



Jede Einzelanweisung belegt genau eine Zeile.

Algo: Deklaration und Zuweisung

<Deklaration>



DEKLARIERE a, b

<Zuweisung>



SETZE a AUF a - b

Algo: Eingabe und Ausgabe

<Eingabe>



LIES a EIN

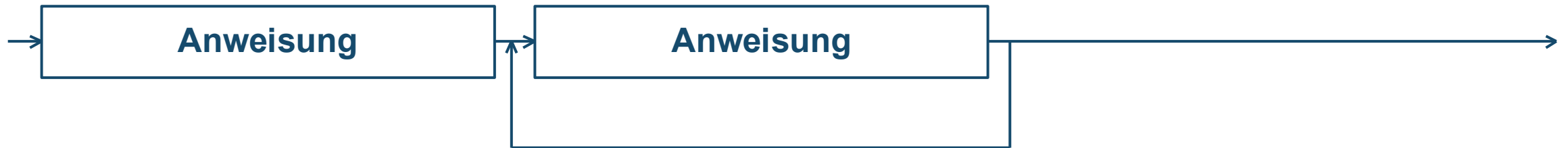
<Ausgabe>



GIB a AUS

Alle Werte sind Ganzzahlen. Ausgaben enthalten keine Zeichenketten.

Algo: <Sequenz>

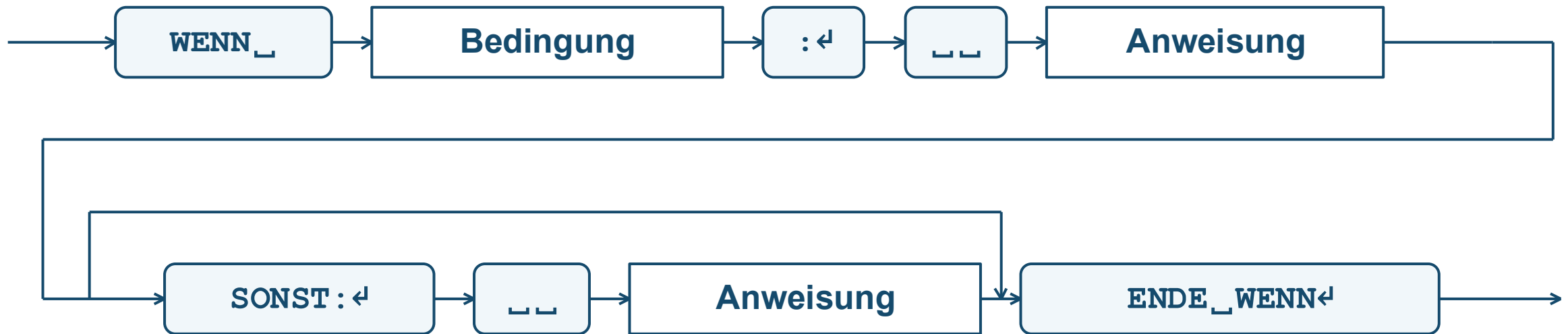


Mindestens zwei Anweisungen stehen hintereinander.
Der Rückweg ergänzt beliebig viele weitere Anweisungen.

Jede davon darf wieder eine Sequenz, Selektion oder Iteration sein.

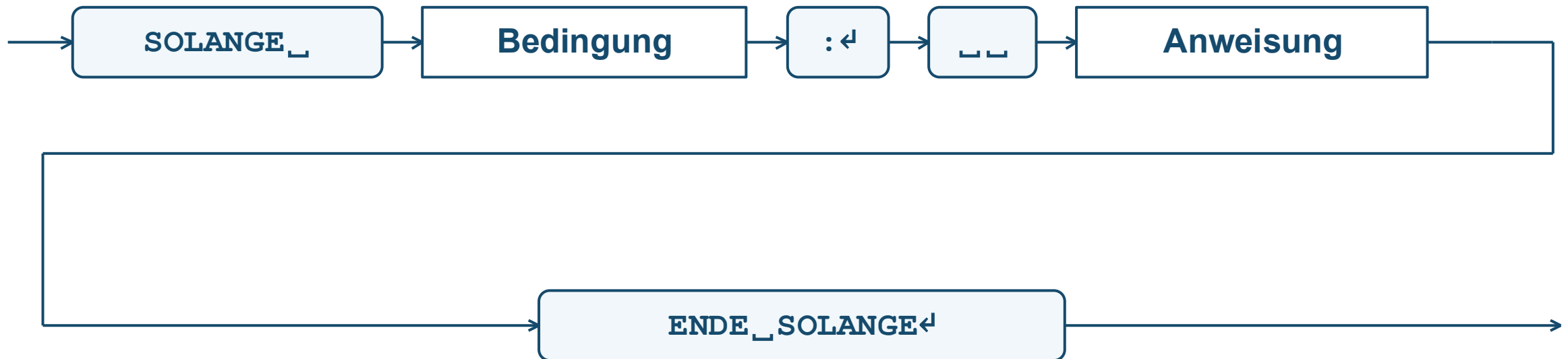
Die Zeilenumbrüche liefern die eingesetzten Anweisungen selbst.

Algo: <Selektion>



SONST ist optional. _ _ gilt vor jeder Zeile der jeweils eingesetzten Anweisung.

Algo: <Iteration>



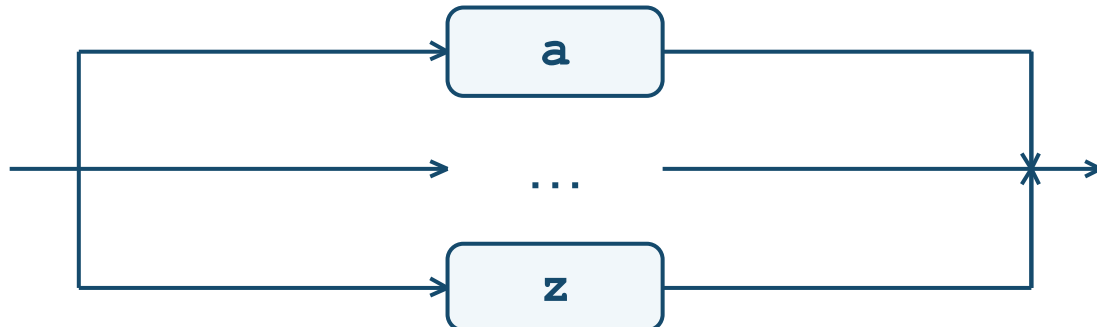
Der Rumpf ist eine Anweisung. Ihre Zeilen stehen jeweils zwei Leerzeichen tiefer.

Algo: Bezeichner und Buchstaben

<Bezeichner>



<Buchstabe>

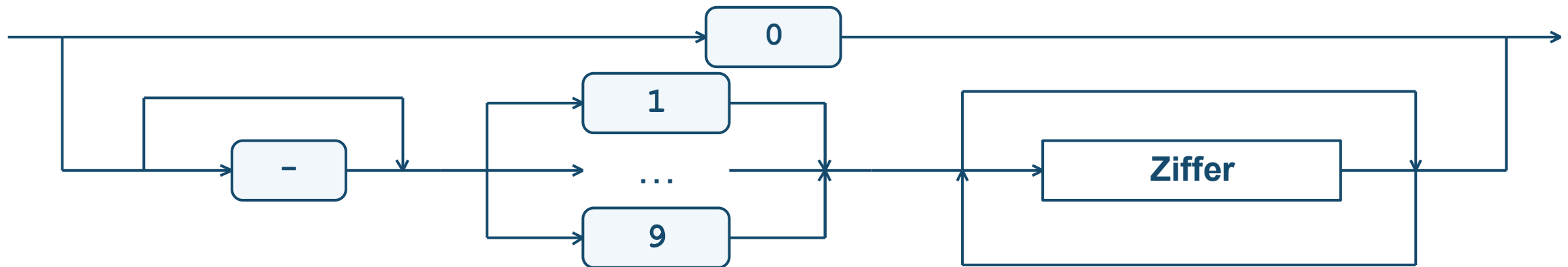


Genau einen Kleinbuchstaben wählen.
Die Punkte stehen für die gleich
aufgebauten Zweige b bis y.

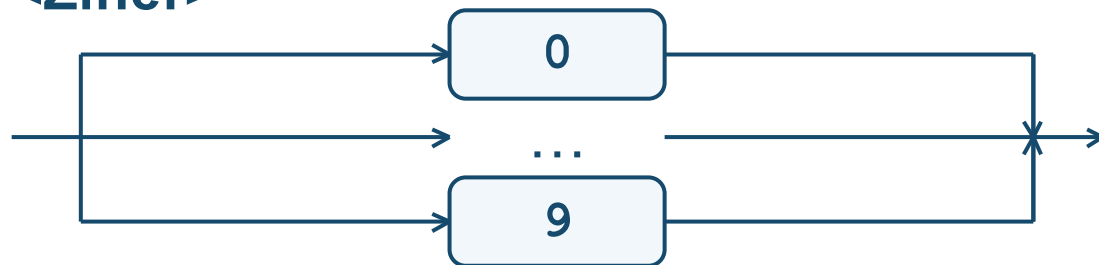
Beispiele: a, b, anzahl. Keine Großbuchstaben, Ziffern oder Unterstriche.

Algo: Ganzzahlen und Ziffern

<Ganzzahl>



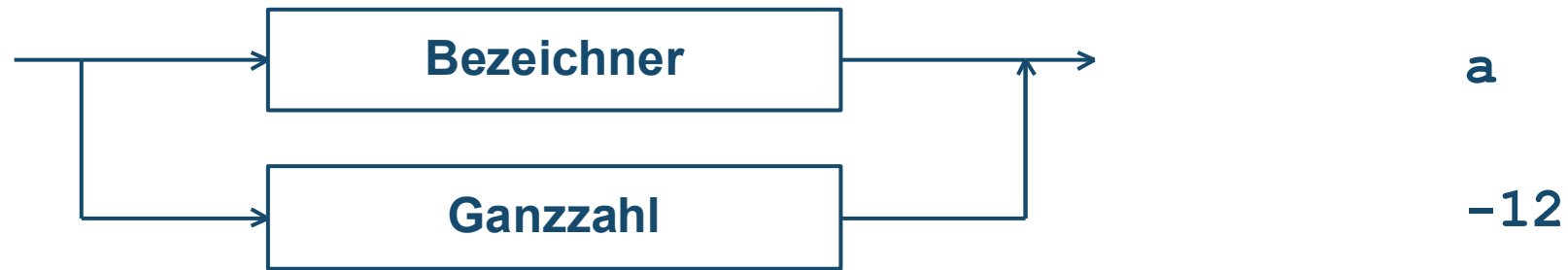
<Ziffer>



Jeder ODER-Pfad liefert eine Ziffer.
Die Punkte stehen für die
ausgelassenen Ziffernzweige.

Algo: Werte und Ausdrücke

<Wert>



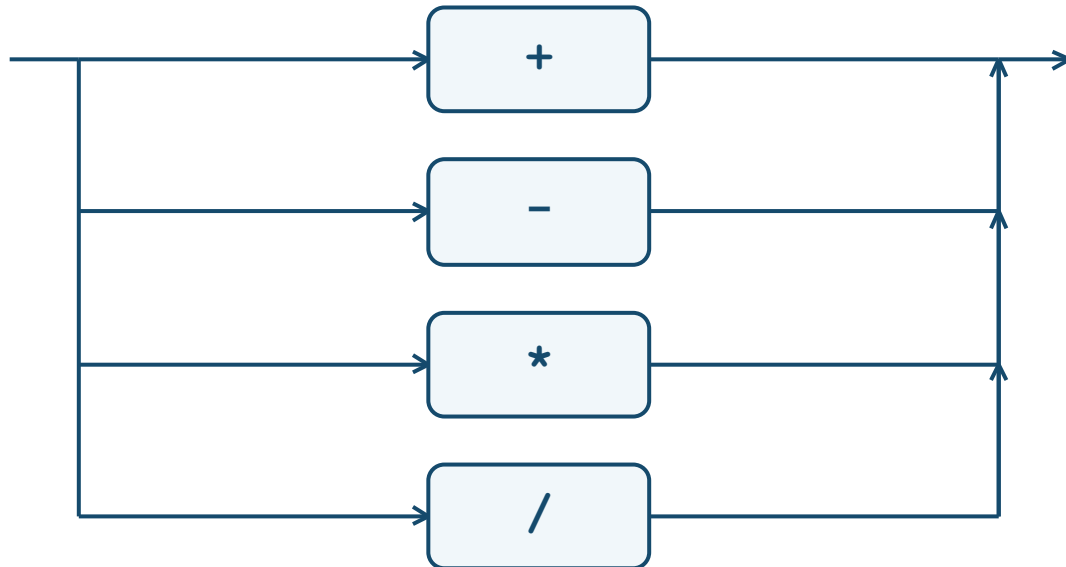
<Ausdruck>



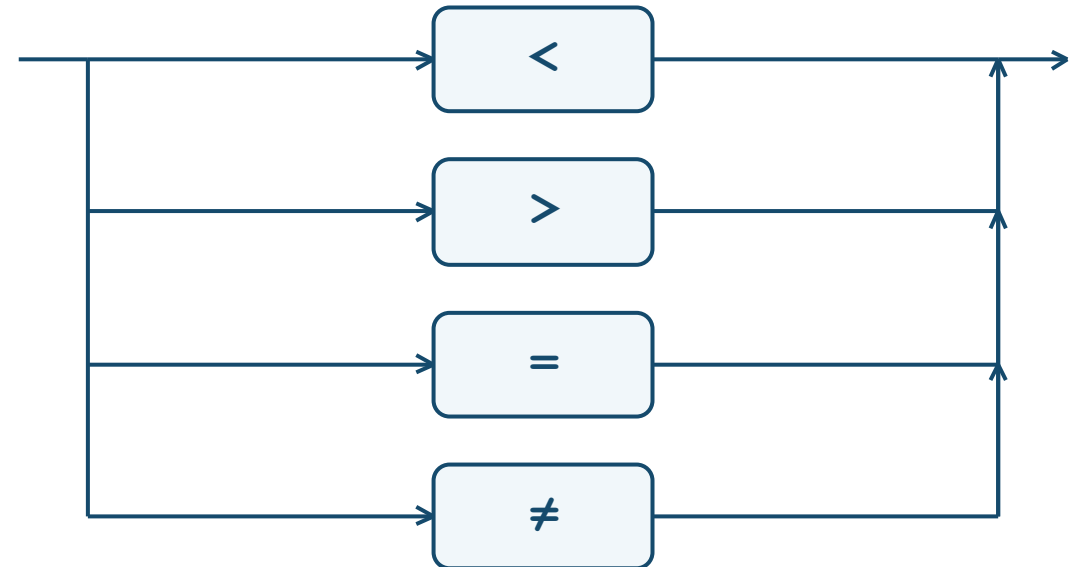
Erlaubt: a, 7, a - b, 7 / 2. Nicht erlaubt: a + b + 1 oder (a + b).

Algo: Rechen- und Vergleichsoperatoren

<Rechenoperator>



<Vergleichsoperator>



Ganzzahlige Division: $7 / 2$ ergibt 3, $-7 / 2$ ergibt -3. Nachkommastellen entfallen.

Algo: <Bedingung>



`a ≠ b`

`anzahl > 0`

`7 < 12`

Zwei Ganzzahlwerte vergleichen.
Das Ergebnis ist wahr oder falsch.

**Bedingungen steuern WENN und
SOLANGE.**

Unser Ziel: den ggT in Algo erzeugen

```
DEKLARIERE a, b
LIES a EIN
LIES b EIN
SOLANGE a ≠ b:
    WENN a > b:
        SETZE a AUF a - b
    SONST:
        SETZE b AUF b - a
    ENDE WENN
ENDE SOLANGE
GIB a AUS
```

Bekannt aus dem letzten Kapitel

**Können wir genau dieses
Programm mit unserer Algo-
Grammatik erzeugen?**

Start: <Anweisung>

Wir ersetzen die Hilfssymbole Schritt
für Schritt, bis nur noch die Zeichen
des Programms übrig sind.

ggT ableiten: Schritte 0–5

- 0 <Anweisung>
- 1 ⊢ <Sequenz>
- 2 ⊢ <Anweisung><Anweisung><Anweisung><Anweisung><Anweisung>
- 3 ⊢ <Einzelanweisung><Anweisung><Anweisung><Anweisung><Anweisung>
- 4 ⊢ <Deklaration><Anweisung><Anweisung><Anweisung><Anweisung>
- 5 ⊢ **DEKLARIERE** <Bezeichner>, <Bezeichner>
 <Anweisung><Anweisung><Anweisung><Anweisung>

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 6–9

- 6 $\vdash$ **DEKLARIERE** <Buchstabe>, <Bezeichner>
 <Anweisung><Anweisung><Anweisung><Anweisung>
- 7 $\vdash$ **DEKLARIERE** a, <Bezeichner>
 <Anweisung><Anweisung><Anweisung><Anweisung>
- 8 $\vdash$ **DEKLARIERE** a, <Buchstabe>
 <Anweisung><Anweisung><Anweisung><Anweisung>
- 9 $\vdash$ **DEKLARIERE** a, b
 <Anweisung><Anweisung><Anweisung><Anweisung>

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 10–13

- 10 $\vdash$ DEKLARIERE a, b
 <Einzelanweisung><Anweisung><Anweisung><Anweisung>
- 11 $\vdash$ DEKLARIERE a, b
 <Eingabe><Anweisung><Anweisung><Anweisung>
- 12 $\vdash$ DEKLARIERE a, b
 LIES <Bezeichner> EIN
 <Anweisung><Anweisung><Anweisung>
- 13 $\vdash$ DEKLARIERE a, b
 LIES <Buchstabe> EIN
 <Anweisung><Anweisung><Anweisung>

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 14–16

- 14 $\vdash$ DEKLARIERE a, b
LIES a EIN
<Anweisung><Anweisung><Anweisung>
- 15 $\vdash$ DEKLARIERE a, b
LIES a EIN
<Einzelanweisung><Anweisung><Anweisung>
- 16 $\vdash$ DEKLARIERE a, b
LIES a EIN
<Eingabe><Anweisung><Anweisung>

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 17–18

```
17  ⊢  DEKLARIERE a, b  
      LIES a EIN  
      LIES <Bezeichner> EIN  
      <Anweisung><Anweisung>
```

```
18  ⊢  DEKLARIERE a, b  
      LIES a EIN  
      LIES <Buchstabe> EIN  
      <Anweisung><Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 19–20

```
19  ⊢  DEKLARIERE a, b  
      LIES a EIN  
      LIES b EIN  
      <Anweisung><Anweisung>
```

```
20  ⊢  DEKLARIERE a, b  
      LIES a EIN  
      LIES b EIN  
      <Iteration><Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 21–22

```
21  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE <Bedingung>:
        <Anweisung> ENDE SOLANGE
      <Anweisung>

22  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE <Wert> <Vergleichsoperator> <Wert>:
        <Anweisung> ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 23–24

```
23  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE <Bezeichner> <Vergleichsoperator> <Wert>:
        <Anweisung> ENDE SOLANGE
      <Anweisung>

24  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE <Buchstabe> <Vergleichsoperator> <Wert>:
        <Anweisung> ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 25–26

```
25  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a <Vergleichsoperator> <Wert>:
        <Anweisung>ENDE SOLANGE
      <Anweisung>
```

```
26  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ <Wert>:
        <Anweisung>ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 27–28

```
27  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ <Bezeichner>:
        <Anweisung> ENDE SOLANGE
      <Anweisung>

28  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ <Buchstabe>:
        <Anweisung> ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritte 29–30

```
29  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        <Anweisung> ENDE SOLANGE
      <Anweisung>

30  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        <Selektion> ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 31

```
31  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
          WENN <Bedingung>:
              <Anweisung>  SONST:
                  <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 32

```
32  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
          WENN <Wert> <Vergleichsoperator> <Wert>:
              <Anweisung>  SONST:
                  <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 33

```
33  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
          WENN <Bezeichner> <Vergleichsoperator> <Wert>:
              <Anweisung>  SONST:
              <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 34

```
34  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
          WENN <Buchstabe> <Vergleichsoperator> <Wert>:
              <Anweisung>  SONST:
              <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 35

```
35  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
          WENN a <Vergleichsoperator> <Wert>:
              <Anweisung>  SONST:
              <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 36

```
36  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > <Wert>:
          <Anweisung>  SONST:
          <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 37

```
37  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
          WENN a > <Bezeichner>:
              <Anweisung>  SONST:
                  <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 38

```
38  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > <Buchstabe>:
          <Anweisung>  SONST:
            <Anweisung> ENDE WENN
        ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 39

```
39  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          <Anweisung>  SONST:
          <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 40

```
40  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          <Einzelanweisung>  SONST:
            <Anweisung>  ENDE WENN
        ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 41

```
41  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          <Zuweisung>  SONST:
          <Anweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 42

```
42  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE <Bezeichner> AUF <Ausdruck>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 43

```
43  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE <Buchstabe> AUF <Ausdruck>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 44

```
44  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF <Ausdruck>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 45

```
45  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF <Wert> <Rechenoperator> <Wert>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 46

```
46  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF <Bezeichner> <Rechenoperator> <Wert>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 47

```
47  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF <Buchstabe> <Rechenoperator> <Wert>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 48

```
48  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a <Rechenoperator> <Wert>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 49

```
49  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - <Wert>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 50

```
50  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - <Bezeichner>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 51

```
51  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - <Buchstabe>
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 52

```
52  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          <Anweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 53

```
53  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          <Einzelanweisung>  ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 54

```
54  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          <Zuweisung> ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 55

```
55  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE <Bezeichner> AUF <Ausdruck>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 56

```
56  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE <Buchstabe> AUF <Ausdruck>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 57

```
57  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF <Ausdruck>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 58

```
58  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF <Wert> <Rechenoperator> <Wert>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 59

```
59  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF <Bezeichner> <Rechenoperator> <Wert>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 60

```
60  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF <Buchstabe> <Rechenoperator> <Wert>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 61

```
61  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b <Rechenoperator> <Wert>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 62

```
62  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - <Wert>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 63

```
63  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - <Bezeichner>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 64

```
64  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - <Buchstabe>
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 65

```
65  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - a
        ENDE WENN
      ENDE SOLANGE
      <Anweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 66

```
66  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - a
        ENDE WENN
      ENDE SOLANGE
      <Einzelanweisung>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 67

```
67  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - a
        ENDE WENN
      ENDE SOLANGE
      <Ausgabe>
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 68

```
68  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - a
        ENDE WENN
      ENDE SOLANGE
      GIB <Wert> AUS
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 69

```
69  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - a
        ENDE WENN
      ENDE SOLANGE
      GIB <Bezeichner> AUS
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 70

```
70  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - a
        ENDE WENN
      ENDE SOLANGE
      GIB <Buchstabe> AUS
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

ggT ableiten: Schritt 71

```
71  ⊢  DEKLARIERE a, b
      LIES a EIN
      LIES b EIN
      SOLANGE a ≠ b:
        WENN a > b:
          SETZE a AUF a - b
        SONST:
          SETZE b AUF b - a
        ENDE WENN
      ENDE SOLANGE
      GIB a AUS
```

Farbig: gerade verändert. Offene Anweisungen liefern ihre Zeilenumbrüche erst beim Ersetzen.

Das Ergebnis: unser ggT-Programm in Algo

```
DEKLARIERE a, b
LIES a EIN
LIES b EIN
SOLANGE a  $\neq$  b:
    WENN a > b:
        SETZE a AUF a - b
    SONST:
        SETZE b AUF b - a
    ENDE WENN
ENDE SOLANGE
GIB a AUS
```

Alle Hilfssymbole sind ersetzt.

Übrig bleibt genau das Programm, das wir erzeugen wollten. Es gehört zur Sprache Algo.

Für zwei positive Ganzzahlen berechnet es den ggT.
Beispiel: 18 und 12 ergeben 6.

Lernziele: Grammatiken selbst anwenden

Das trainieren Sie in der anschließenden Übung:

Grammatiken entwerfen

Kleine Sprachen durch Typ-2-Produktionsregeln, BNF und EBNF beschreiben.

Wörter herleiten

Vom Startsymbol ausgehen und durch konkrete Ableitungsschritte die Zugehörigkeit zeigen.

Regelformen einordnen

Für alle Regeln einer Grammatik Typ 0 bis 3 prüfen.
Bei Typ 3 auch: links- oder rechtslinear?

Ziel: selbst Grammatikregeln aufstellen und konkrete Wörter daraus herleiten.